

CFAI D'Auvergne, Cournon-d'Auvergne

RAPPORT DE PROJET

Revue Finale

Brevet de Technicien Supérieur
Cybersécurité, Informatique et réseaux, Électronique

INTITULÉ DU PROJET

**Développement et déploiement d'une interface web de centralisation
et de gestion de la documentation technique du banc de test KNX**

CANDIDAT

Théo Magnin
Apprenti à la Banque de France

EN BINÔME AVEC

Maxence Nicoletti

SOUTENANCE

8 septembre 2026

Sommaire

1. Introduction et contexte	3	6.2 Administration et sécurisation système	29
1.1 Contexte et problématique du banc de test KNX	4	6.3 Déploiement et durcissement de MariaDB	29
1.2 Les publics cibles et leurs besoins	4	6.4 Architecture réseau : OPNsense et segmentation	30
1.3 Objectifs et périmètre du projet	5	7. Tests et recette	32
2. Cahier des charges	6	7.1 Démarche de validation	33
2.1 Synthèse des exigences fonctionnelles	7	7.2 Synthèse du cahier de recette	33
2.2 Exigences d'infrastructure, de sécurité et d'ergonomie	7	7.3 État factuel du système au 8 septembre 2026	34
2.3 Arbitrages et fonctionnalités reportées	8	8. Difficultés et solutions	35
3. Analyse et conception	9	8.1 Évolution du schéma de base sans outil de migration	36
3.1 Architecture applicative	10	8.2 Incompatibilité de l'environnement Python entre macOS et Ubuntu	36
3.2 Modélisation des données	11	8.3 Accessibilité réseau du serveur applicatif	37
3.3 Conception ergonomique et charte différenciée	13	8.4 Panne matérielle du routeur et diagnostic par élimination	38
4. Veille technologique et choix techniques	16	8.5 Mise en place tardive de la gestion de version	39
4.1 Comparatif des frameworks web	17	9. Gestion de projet	40
4.2 Base de données et ORM	17	9.1 Jalons et suivi	41
4.3 Système d'exploitation serveur	18	9.2 Gestion de version et conventions	41
4.4 Équipement réseau : OPNsense face au routeur matériel	18	9.3 Analyse des écarts entre le prévu et le réalisé	42
5. Réalisation applicative	20	10. Compétences BTS CIEL mobilisées	43
5.1 Socle CRUD des fiches modules	21	10.1 Ce que le projet m'a concrètement appris	44
5.2 Authentification et cloisonnement des droits	22	11. Conclusion	46
5.3 Gestion des comptes utilisateurs	23	11.1 Bilan du projet	47
5.4 Fichiers joints : téléversement, classement, stockage	24	11.2 Bilan personnel	47
5.5 Recherche et espace de consultation	25	12. Glossaire technique	48
5.6 Journalisation et tableaux de bord	26	13. Annexes	50
6. Infrastructure serveur et réseau	28	Annexe 1 : Cahier des charges	51
6.1 Matériel et découpage du stockage	29	Annexe 2 : Cahier de recette	51

Introduction et contexte

Le CFAI d'Auvergne, implanté sur le site de Cournon-d'Auvergne, est un centre de formation d'apprentis dédié aux métiers de l'industrie et du numérique. Le BTS Cybersécurité, Informatique et réseaux, Électronique forme des techniciens capables d'intervenir sur l'ensemble de la chaîne du numérique : développement d'applications, administration de systèmes et de réseaux, sécurisation des infrastructures.

En tant qu'apprenti au CFAI, mon parcours se déroule en alternance au sein de la Banque de France. Cette immersion dans un environnement d'infrastructures critiques, où la traçabilité et la

sécurité sont des exigences quotidiennes, a directement influencé la façon dont j'ai abordé ce projet : en particulier le soin apporté au cloisonnement des droits, à la journalisation des actions et au durcissement du serveur.

Le projet consiste à concevoir, développer et déployer une plateforme web centralisée, hébergée sur le réseau local de l'établissement, faisant office de référentiel documentaire unique pour le banc de test KNX. Il mobilise conjointement le développement applicatif, l'administration système et l'architecture réseau.

1.1 Contexte et problématique du banc de test KNX

Les travaux pratiques de domotique et de gestion technique du bâtiment s'appuient sur un support matériel central : le banc de test KNX. KNX est un standard international ouvert (norme ISO/IEC 14543) de contrôle et d'automatisation des bâtiments. Un système KNX repose sur un bus de communication interconnectant des modules autonomes : capteurs en entrée (interrupteurs, détecteurs de présence, sondes), actionneurs en sortie (modules de commutation, variateurs, commandes de volets) et organes système (alimentations de bus, coupleurs, interfaces IP).

Chaque module possède une référence constructeur, une adresse physique, des objets de communication et des paramètres configurables. La documentation associée est donc à la fois abondante et hétérogène : fiches constructeur au format PDF, schémas de câblage, notices de mise en service, guides de travaux pratiques rédigés par les enseignants.

La problématique identifiée

Avant ce projet, cette documentation présentait quatre faiblesses :

- 1. Dispersion.** Les documents étaient répartis entre les disques locaux de plusieurs postes du laboratoire, des dossiers partagés non structurés, des clés USB et des classeurs papier.
- 2. Perte de temps en séance.** Un apprenti devait souvent chercher les caractéristiques d'un module avant de pouvoir commencer sa manipulation, ou solliciter un formateur.
- 3. Absence de point de mise à jour unique.** Un formateur ajoutant un module au banc n'avait pas d'endroit évident où en publier la documentation.
- 4. Risque matériel.** L'absence d'accès immédiat au schéma de raccordement correct augmente le risque d'erreur de câblage, et donc de détérioration des modules.

Le besoin exprimé par l'équipe pédagogique lors de la première revue de projet était donc de concevoir, développer et déployer une plateforme web centralisée, hébergée sur le réseau local de l'établissement, faisant office de référentiel documentaire unique pour le banc KNX.

1.2 Les publics cibles et leurs besoins

Deux profils d'utilisateurs ont été identifiés dès l'analyse du besoin. Leurs attentes structurent l'ensemble de l'application, jusqu'à son ergonomie.

Espace apprenti, charte bleue	Espace enseignant, charte ambre
Consultation en lecture seule Recherche rapide par nom de module Visualisation des schémas et notices Aucun contrôle d'édition affiché	Création, modification, suppression des fiches Téléversement de fichiers joints Gestion des comptes utilisateurs Consultation du journal d'activité

Les apprentis utilisent la plateforme comme un outil de consultation pendant leurs travaux pratiques : ils ont besoin d'un accès rapide, d'une recherche efficace et d'une interface épurée dont tout contrôle d'édition est absent, afin d'éviter toute modification involontaire. Les enseignants sont responsables du contenu et des accès : ils gèrent le catalogue, joignent les documents, administrent les comptes des apprentis et disposent d'un suivi des modifications.

1.3 Objectifs et périmètre du projet

Axe 1 : Développement applicatif

Concevoir une application web en Python avec le framework Flask 3.1, adossée à une base MariaDB pilotée par un ORM. L'application est organisée selon le patron de l'application factory, avec une séparation nette entre les modèles de données, les contrôleurs et les vues.

Axe 2 : Infrastructure système et sécurité

Installer, configurer et sécuriser un serveur physique dédié sous Ubuntu Server : partitionnement LVM, administration à distance par SSH sans connexion root directe, durcissement du moteur MariaDB et création d'un compte applicatif aux droits restreints à la seule base du projet.

Axe 3 : Architecture et segmentation réseau

Mettre en place un réseau local stable desservant la salle de travaux pratiques. À la suite de la panne du routeur initialement prévu, l'architecture a été reconstruite autour d'un pare-feu et routeur OPNsense installé sur une machine physique dédiée, associé à un switch administrable. Le réseau est segmenté en VLAN 802.1Q séparant le serveur, les postes enseignants et les postes apprentis.

Axe 4 : Qualification et transparence de l'état livré

Conduire la recette au travers d'un cahier de 30 points de contrôle, et rendre compte de l'état exact du système à la date du rendu : ce qui est opérationnel, ce qui a été vérifié en environnement isolé mais reste à confirmer en conditions réelles, et ce qui n'a pas été réalisé. Cette transparence est un choix assumé et constitue, à nos yeux, une part du travail d'ingénierie.

Cahier des charges

Le cahier des charges validé lors des premières revues regroupe les services que l'application doit rendre à chaque catégorie d'utilisateur, ainsi que les contraintes d'infrastructure, de sécurité et d'ergonomie auxquelles le système doit satisfaire.

Il se termine sur deux arbitrages assumés : les fonctionnalités qui, faute de temps, ont été

explicitement reportées plutôt que livrées à moitié.

Les exigences présentées ici sont celles qui ont été validées avec l'équipe pédagogique, et qui servent de référence à la recette exposée au chapitre 7. Chacune est reprise, point par point, dans le cahier de recette joint en annexe.

2.1 Synthèse des exigences fonctionnelles

Réf.	Besoin fonctionnel	Description	Priorité
EF-01	Consultation du catalogue	Accès à la liste des modules KNX, avec recherche par nom et affichage par catégorie.	Indispensable
EF-02	Socle CRUD	Création, consultation, modification et suppression des fiches modules par les enseignants.	Indispensable
EF-03	Fichiers joints	Téléversement de photos, vidéos et documents, classés automatiquement et rattachés à une fiche.	Indispensable
EF-04	Authentification à rôles	Contrôle d'accès différencié entre apprenti, enseignant et administrateur.	Indispensable
EF-05	Gestion des comptes	Création, suppression et réinitialisation de mot de passe, dans les limites du rôle courant.	Important
EF-06	Journalisation	Enregistrement horodaté de chaque connexion, création, modification et suppression.	Important
EF-07	Tableaux de bord	Indicateurs et graphiques sur l'état de la base documentaire et l'activité.	Confort

2.2 Exigences d'infrastructure, de sécurité et d'ergonomie

Infrastructure

- Hébergement sur un serveur physique dédié de l'établissement, sous Ubuntu Server.
- Administration distante exclusivement par SSH, sans connexion directe au compte root.
- Segmentation du réseau par VLAN, gérée par un pare-feu dédié, afin de séparer le serveur, les postes enseignants et les postes apprentis.
- Distribution automatique des adresses par DHCP sur chacun des segments desservant des postes.

Sécurité applicative

- Principe du moindre privilège : l'application accède à MariaDB via un compte dédié, restreint à sa seule base, jamais avec le compte root.
- Cloisonnement vérifié côté serveur : masquer un bouton dans l'interface ne constitue pas une protection. Toute requête vers une URL protégée doit être rejetée par le serveur, y compris lorsqu'elle est saisie directement dans le navigateur.
- Assainissement et unicité des fichiers téléversés, pour prévenir l'écrasement de fichiers et les tentatives de traversée de répertoire.
- Aucune requête SQL construite par concaténation de chaînes issues d'une saisie utilisateur.

Ergonomie

- Différenciation visuelle par rôle : charte bleue pour la consultation, charte ambre pour la gestion.
- Toute action destructrice passe par une étape de confirmation explicite.

2.3 Arbitrages et fonctionnalités reportées

Le calendrier du projet étant contraint, des arbitrages ont été nécessaires. Deux exigences ont été explicitement reportées.

Sauvegardes automatiques de la base de données, non réalisé

Aucun mécanisme d'export programmé ni d'archivage n'a été développé. L'effort a été porté en priorité sur la fiabilité du socle applicatif, le cloisonnement des droits et la reconstruction de l'infrastructure réseau après la panne matérielle.

Chiffrement des échanges en HTTPS, non réalisé

L'application est servie en HTTP sur le réseau local. La génération de certificats et la mise en place d'un serveur mandataire inverse n'ont pas été traitées dans le temps imparti.

Ces deux points sont des manques réels, et non des choix de conception. Ils sont assumés comme tels : face à un délai court, nous avons préféré livrer un cœur applicatif fiable et une infrastructure saine plutôt qu'un plus grand nombre de fonctionnalités partiellement abouties. Ils constituent les deux premières évolutions à traiter.

Analyse et conception

L'application suit le patron de conception de l'application factory, recommandé par la documentation de Flask. Plutôt que d'instancier l'objet applicatif dans une variable globale, sa création est encapsulée dans une fonction qui charge la configuration, initialise les extensions et enregistre les routes.

Cette organisation présente deux avantages concrets : elle permet d'instancier l'application plusieurs fois avec des configurations différentes,

ce qui est précisément ce qui rend les tests automatisés possibles, et elle évite les importations circulaires entre les modules.

La conception couvre trois volets : l'organisation du code applicatif, la modélisation des données, et l'ergonomie de l'interface. Deux décisions de modélisation sont explicitées en détail, parce qu'elles rompent volontairement avec les règles habituelles de normalisation.

3.1 Architecture applicative

```
app/___init___py – application factory & connexion base de données

from flask import Flask
from flask_sqlalchemy import SQLAlchemy
from config import Config

# On crée l'objet "db" ici, hors de create_app, pour pouvoir l'importer
# facilement depuis d'autres fichiers (comme models.py).
db = SQLAlchemy()

def create_app():
    app = Flask(__name__)
    app.config.from_object(Config)

    # On lie l'objet db à l'application Flask.
    db.init_app(app)

    from app.routes import main
    app.register_blueprint(main)

    return app
```

Figure 3.1 — Fonction `create_app()` : chargement de la configuration, initialisation de l'ORM et enregistrement du blueprint.

Le code est réparti en trois responsabilités distinctes :

- Les **modèles** (`app/models.py`) définissent les classes Python cartographiées sur les tables MariaDB par l'ORM. Ils portent la structure des données ainsi que quelques méthodes utilitaires, comme le hachage et la vérification des mots de passe.
- Les **contrôleurs** (`app/routes.py`) reçoivent les requêtes HTTP, appliquent les contrôles de droits, interrogent les modèles et choisissent le gabarit à afficher. Ils sont regroupés dans un blueprint unique nommé `main`, enregistré par l'application factory.
- Les **vues** (`app/templates/`) sont des gabarits Jinja2 héritant tous d'un gabarit de base commun, `base.html`, qui porte la structure de page et la barre de navigation adaptée au rôle connecté.

Le choix d'un blueprint unique plutôt que de plusieurs modules séparés correspond à la taille du projet : une vingtaine de routes, un domaine fonctionnel homogène. Un découpage en plusieurs blueprints constituerait l'évolution naturelle si l'application devait s'étendre, par exemple pour couvrir d'autres bancs de test de l'établissement.

3.2 Modélisation des données

La base de données `knx_doc` s'organise autour de quatre tables. La conception vise l'absence de redondance et la préservation de la traçabilité.

Table	Colonne	Type	Rôle
modules	id	INT, clé primaire	Identifiant du module KNX
	nom	VARCHAR(100), non nul	Désignation du module
	description	TEXT	Description technique et indications de câblage
	categorie	VARCHAR(50)	Famille du module (capteur, actionneur, système)
	date_maj	DATETIME	Horodatage automatique de la dernière modification
utilisateurs	id	INT, clé primaire	Identifiant du compte
	identifiant	VARCHAR(50), unique	Identifiant de connexion
	nom_complet	VARCHAR(100)	Nom affiché
	mot_de_passe_hash	VARCHAR(255)	Empreinte du mot de passe, jamais le mot de passe
	role	VARCHAR(20), non nul	apprenti, enseignant ou admin
	date_creation	DATETIME	Date de création du compte
fichiers_joints	id	INT, clé primaire	Identifiant du fichier
	module_id	INT, clé étrangère	Module auquel le fichier est rattaché
	nom_original	VARCHAR(255)	Nom du fichier tel que fourni par l'utilisateur
	nom_stocke	VARCHAR(255)	Nom unique réellement utilisé sur le disque
	type_fichier	VARCHAR(20)	photo, video ou document
	taille_octets	INT	Taille du fichier
	date_ajout	DATETIME	Horodatage du téléversement
journal_activite	id	INT, clé primaire	Identifiant de l'entrée de journal
	date_action	DATETIME	Horodatage de l'action
	identifiant_utilisateur	VARCHAR(50)	Identifiant de l'auteur, copié en texte
	role_utilisateur	VARCHAR(20)	Rôle de l'auteur au moment de l'action
	action	VARCHAR(50), non nul	Nature de l'action (connexion, creation_module...)
	cible	VARCHAR(255)	Élément concerné
	module_id	INT, non contraint	Module concerné, sans clé étrangère (voir ci-dessous)

```
app/models.py — modèle de données Module

# Chaque classe ici correspond à une table SQL, chaque attribut à une colonne.
# C'est le principe de l'ORM : on manipule des objets Python, SQLAlchemy
# traduit ça en requêtes SQL derrière.

from app import db

class Module(db.Model):
    __tablename__ = 'modules'

    id = db.Column(db.Integer, primary_key=True)
    nom = db.Column(db.String(100), nullable=False)
    description = db.Column(db.Text)
    categorie = db.Column(db.String(50))
    date_maj = db.Column(db.DateTime, server_default=db.func.now(), onupdate=db.func.now())

    def __repr__(self):
        return f"<Module {self.nom}>"
```

Figure 3.2 — Définition du modèle `Module` : chaque attribut de classe correspond à une colonne SQL.

Deux décisions de conception qui méritent d'être expliquées

La première concerne la relation entre un module et ses fichiers joints. Elle est déclarée au niveau de l'ORM avec l'option `cascade='all, delete-orphan'` : lorsqu'un module est supprimé, les enregistrements de ses fichiers joints le sont également. La suppression est donc gérée par l'application plutôt que déléguée au moteur de base de données, ce qui permet de supprimer au passage les fichiers physiques correspondants sur le disque, opération dont MariaDB ne pourrait pas se charger.

La seconde concerne le journal d'activité, et elle est volontairement contre-intuitive. Les colonnes `identifiant_utilisateur` et `role_utilisateur` dupliquent en texte des informations qui existent déjà dans la table `utilisateurs`, et la colonne `module_id` ne porte aucune clé étrangère. Ce choix, qui rompt délibérément avec la normalisation, répond à une exigence de traçabilité : un journal doit rester lisible même après la suppression du compte ou du module concerné. Si l'on avait posé une clé étrangère, la suppression d'un compte aurait soit interdit la suppression, soit vidé la référence, soit effacé les lignes de journal, trois comportements inacceptables pour un journal d'audit. En dupliquant l'information au moment de l'écriture, on conserve un historique exploitable dans la durée.

3.3 Conception ergonomique et charte différenciée

Pour que l'utilisateur identifie immédiatement le niveau de droits dont il dispose, l'interface est déclinée en deux chartes chromatiques, portées par une feuille de style commune et un gabarit de page partagé.

- **Espace de consultation (apprenti)** : dominante bleue. Seules la recherche et la consultation sont proposées ; les contrôles d'édition ne sont pas seulement désactivés, ils sont absents de la page.
- **Espace de gestion (enseignant et administrateur)** : dominante ambre. La bannière change de couleur, un rappel du rôle actif est affiché, et les actions de création, modification et suppression apparaissent, la suppression en rouge, pour la distinguer nettement des autres.

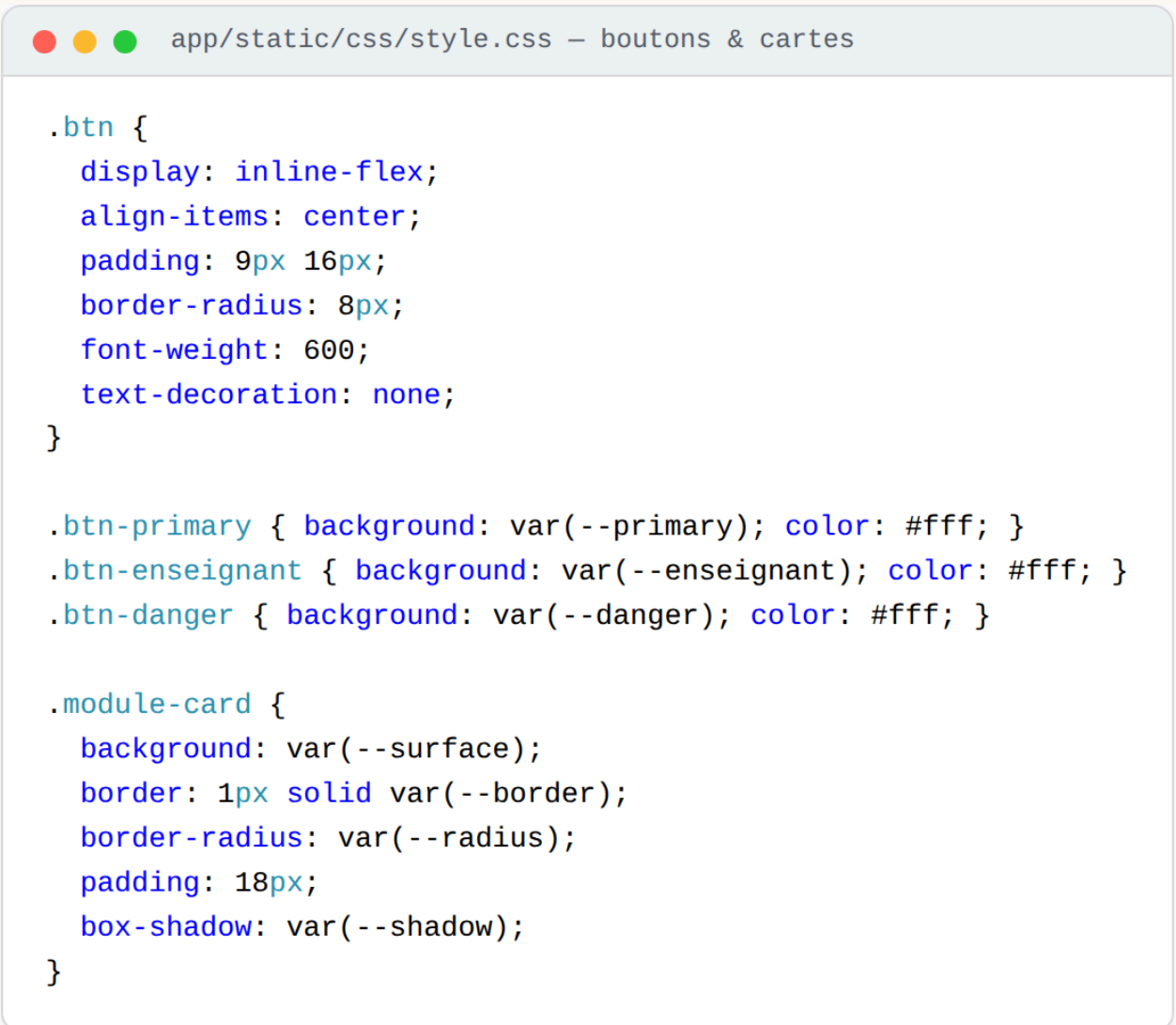


Figure 3.3 — Feuille de style : déclinaison des boutons et des couleurs selon le rôle.

Toute suppression passe par une page de confirmation intermédiaire rappelant explicitement l'élément concerné. Ce n'est pas une simple précaution d'ergonomie : dans le contexte d'une salle de travaux pratiques, où plusieurs utilisateurs se succèdent rapidement sur les mêmes postes, la suppression accidentelle d'une fiche documentée constitue une perte réelle de travail.

```
app/templates/modules_liste.html – grille de cartes

{% extends "base.html" %}
{% block content %}
  <div class="page-header">
    <h1>Documentation des modules KNX</h1>
    <a class="btn btn-enseignant" href="{{ url_for('main.nouveau_module') }}">+ Ajouter un module</a>
  </div>
  {% if modules %}
    <div class="module-grid">
      {% for module in modules %}
        <div class="module-card">
          <span class="badge badge-categorie">{{ module.categorie }}</span>
          <h3><a href="{{ url_for('main.detail_module', module_id=module.id) }}">
            {{ module.nom }}</a></h3>
          <div class="module-actions">
            <a class="btn btn-outline btn-sm" href="...">Consulter</a>
            <a class="btn btn-outline btn-sm" href="...">Modifier</a>
            <a class="btn btn-outline btn-sm" href="...">Supprimer</a>
          </div>
        </div>
      {% endfor %}
    </div>
  {% endif %}
{% endblock %}
```

Figure 3.4 — Gabarit de la liste des modules : présentation en cartes, actions contextuelles selon le rôle.

Veille technologique et choix techniques

Chaque brique technique du projet a fait l'objet d'un arbitrage explicite. Ce chapitre expose les options envisagées, les critères retenus et les raisons pour lesquelles une solution a été préférée aux autres.

Il inclut le choix le plus structurant du projet : celui de reconstruire le réseau sur OPNsense

plutôt que d'attendre le remplacement à l'identique du routeur tombé en panne.

Chaque comparatif présenté ici a été mené sur des critères explicites, et conclut sur les raisons concrètes du choix retenu, y compris lorsque la solution écartée aurait techniquement convenu.

4.1 Comparatif des frameworks web

Critère	Flask (Python)	Django (Python)	PHP (natif ou Laravel)
Philosophie	Micro-framework : le minimum, on ajoute ce dont on a besoin	Framework complet, nombreux composants intégrés d'emblée	Très répandu en hébergement mutualisé
Maîtrise du code	Élevée : peu de comportements implicites	Plus faible : beaucoup de conventions imposées	Variable selon le framework retenu
Administration	À construire soi-même	Interface générée automatiquement	À construire ou via un framework
Apprentissage	Rapide	Plus long (ORM, applications, conventions)	Rapide en natif
Adéquation	Bonne : périmètre restreint, besoin de comprendre chaque brique	Surdimensionné pour une vingtaine de routes	Écart avec la stack Python travaillée en formation

Flask a été retenu pour deux raisons. D'abord parce que le périmètre du projet ne justifiait pas l'infrastructure de Django : l'interface d'administration générée automatiquement, qui constitue son principal argument, ne correspondait pas à notre besoin, puisque nous devons précisément concevoir une interface de gestion adaptée à des enseignants et non à des développeurs. Ensuite pour une raison pédagogique assumée : avec Flask, l'authentification, le contrôle des droits et la journalisation doivent être écrits explicitement. Nous les avons donc réellement conçus, plutôt que de les activer par configuration.

4.2 Base de données et ORM

MariaDB a été retenu face à SQLite et PostgreSQL. SQLite, qui stocke la base dans un simple fichier, aurait suffi au volume de données mais gère mal les accès concurrents en écriture : or le scénario d'usage prévoit précisément qu'un enseignant modifie une fiche pendant que plusieurs apprentis consultent le catalogue. PostgreSQL aurait parfaitement convenu, mais MariaDB est le moteur travaillé en formation et disponible dans les dépôts standard d'Ubuntu, ce qui simplifiait le déploiement.

L'accès aux données passe intégralement par l'ORM Flask-SQLAlchemy, appuyé sur le pilote PyMySQL. Ce choix a une conséquence directe en matière de sécurité : les requêtes étant construites par l'ORM et les valeurs transmises sous forme de paramètres, aucune saisie utilisateur n'est jamais concaténée dans une chaîne SQL. La protection contre l'injection SQL découle de l'architecture retenue, et non d'un filtrage ajouté a posteriori, ce qui est nettement plus robuste.

```

config.py — configuration de connexion MariaDB

class Config:
    SQLALCHEMY_DATABASE_URI = 'mysql+pymysql://knx_user:un_mot_de_passe_a_toi@localhost/knx_doc'
    SQLALCHEMY_TRACK_MODIFICATIONS = False
  
```

Figure 4.1 — Chaîne de connexion : le pilote PyMySQL et la base applicative knx_doc.

La limite de cette approche est apparue en cours de projet : Flask-SQLAlchemy sait créer les tables manquantes, mais ne sait pas faire évoluer une table existante. Ce point a provoqué un incident détaillé au chapitre 8 et justifie l'ajout d'un outil de migration comme Alembic pour toute évolution ultérieure du modèle.

4.3 Système d'exploitation serveur

Ubuntu Server a été retenu dans sa version à support long terme. Trois raisons : la durée de support des correctifs de sécurité, adaptée à un équipement destiné à rester en service plusieurs années dans un établissement ; l'absence d'environnement graphique, qui réduit à la fois la consommation de ressources et la surface d'attaque ; et la disponibilité native de LVM dans l'installateur, qui permet de faire évoluer le découpage du stockage sans réinstallation.

4.4 Équipement réseau : OPNsense face au routeur matériel

L'architecture initiale reposait sur un routeur TP-Link ER605. À la suite de sa panne matérielle, détaillée au chapitre 8, la question s'est posée d'attendre un matériel de remplacement identique ou de reconstruire le réseau sur une autre base. Nous avons opté pour l'installation d'OPNsense, distribution open source de pare-feu et de routage, sur une machine physique dédiée.

Critère	Routeur TP-Link ER605	OPNsense sur machine dédiée
Nature	Boîtier propriétaire à micrologiciel fermé	Système open source sur matériel banalisé
Segmentation VLAN	Prise en charge basique, limitée par le micrologiciel	Gestion fine des interfaces et sous-réseaux
Filtrage	Règles élémentaires	Pare-feu à état, règles détaillées par interface
Diagnostic	Voyants et journaux restreints	Journaux détaillés, capture de trafic, console complète
Remplaçabilité	Dépend d'un approvisionnement constructeur	Réinstallable sur toute machine compatible

Ce choix a transformé une contrainte subie en apport pédagogique : il nous a conduits à concevoir explicitement un plan d'adressage et une segmentation en VLAN, là où le matériel d'origine aurait été utilisé dans sa configuration par défaut. Le détail de cette segmentation figure au chapitre 6.

Réalisation applicative

Ce chapitre détaille les six modules fonctionnels de l'application, en s'appuyant sur des extraits du code réellement écrit. Chaque section expose non seulement ce qui a été implémenté, mais aussi le raisonnement qui a guidé l'implémentation.

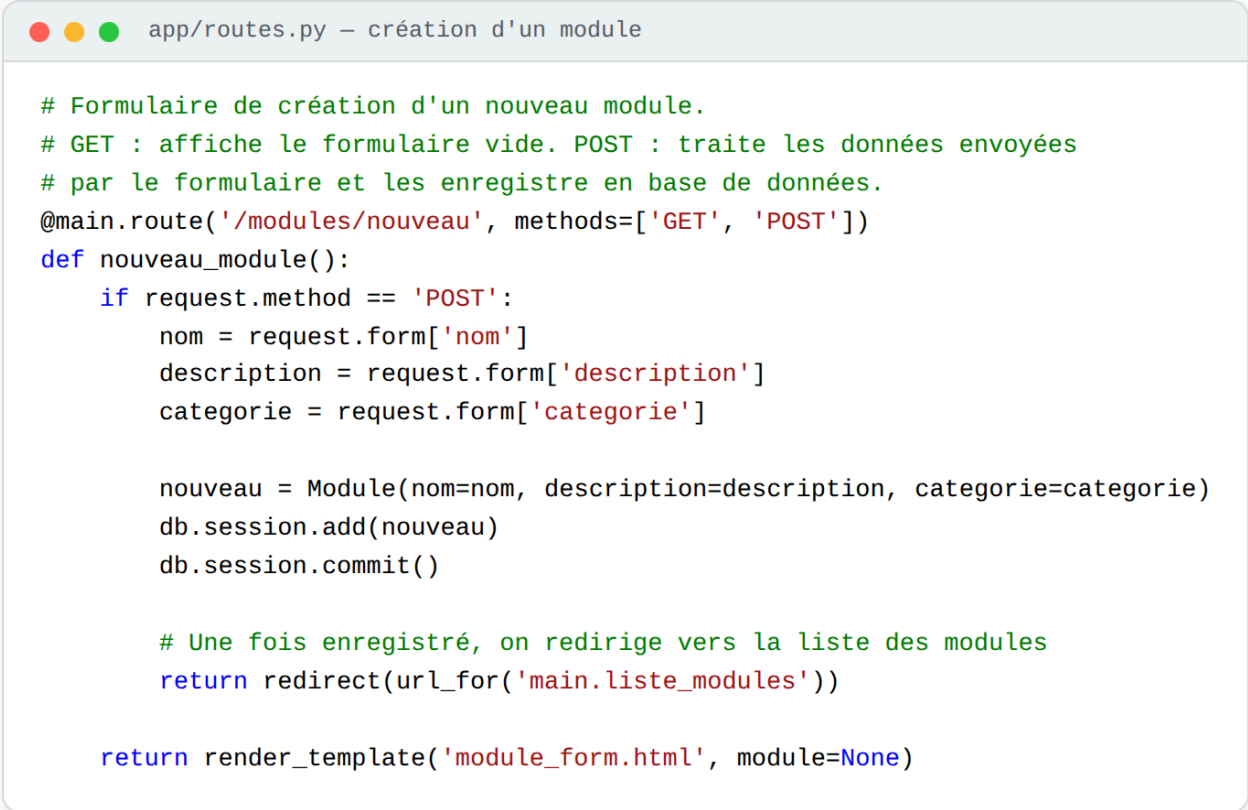
Une attention particulière est portée au contrôle des droits, qui constitue la différence entre une

interface qui paraît protégée et une application qui l'est réellement.

Les extraits de code reproduits dans ce chapitre proviennent directement du dépôt du projet. Ils sont commentés afin que le raisonnement d'implémentation soit lisible, et non seulement le résultat.

5.1 Socle CRUD des fiches modules

Le cœur de l'application est constitué des quatre opérations élémentaires appliquées aux fiches modules. La création et la modification s'appuient sur un même gabarit de formulaire, appelé avec ou sans module existant selon le cas, ce qui évite de dupliquer le code d'affichage.



```

app/routes.py – création d'un module

# Formulaire de création d'un nouveau module.
# GET : affiche le formulaire vide. POST : traite les données envoyées
# par le formulaire et les enregistre en base de données.
@main.route('/modules/nouveau', methods=['GET', 'POST'])
def nouveau_module():
    if request.method == 'POST':
        nom = request.form['nom']
        description = request.form['description']
        categorie = request.form['categorie']

        nouveau = Module(nom=nom, description=description, categorie=categorie)
        db.session.add(nouveau)
        db.session.commit()

        # Une fois enregistré, on redirige vers la liste des modules
        return redirect(url_for('main.liste_modules'))

    return render_template('module_form.html', module=None)

```

Figure 5.1 — Création d'un module : formulaire affiché sur requête GET, données enregistrées sur requête POST.

La suppression fait l'objet d'un traitement particulier. La route accepte à la fois GET et POST : la requête GET affiche une page de confirmation rappelant le module concerné, et seule la requête POST, émise par cette page, déclenche la suppression effective. Un module ne peut donc pas être supprimé par un simple lien, ce qui écarte les suppressions accidentelles.

```

app/routes.py – suppression d'un module

# Suppression d'un module, avec une étape de confirmation intermédiaire
# pour éviter une suppression accidentelle (cf. cahier des charges).
# GET : affiche la page de confirmation. POST : supprime réellement.
@main.route('/modules/<int:module_id>/supprimer', methods=['GET', 'POST'])
def supprimer_module(module_id):
    module = Module.query.get_or_404(module_id)

    if request.method == 'POST':
        db.session.delete(module)
        db.session.commit()
        return redirect(url_for('main.liste_modules'))

    return render_template('module_confirm_suppression.html', module=module)

```

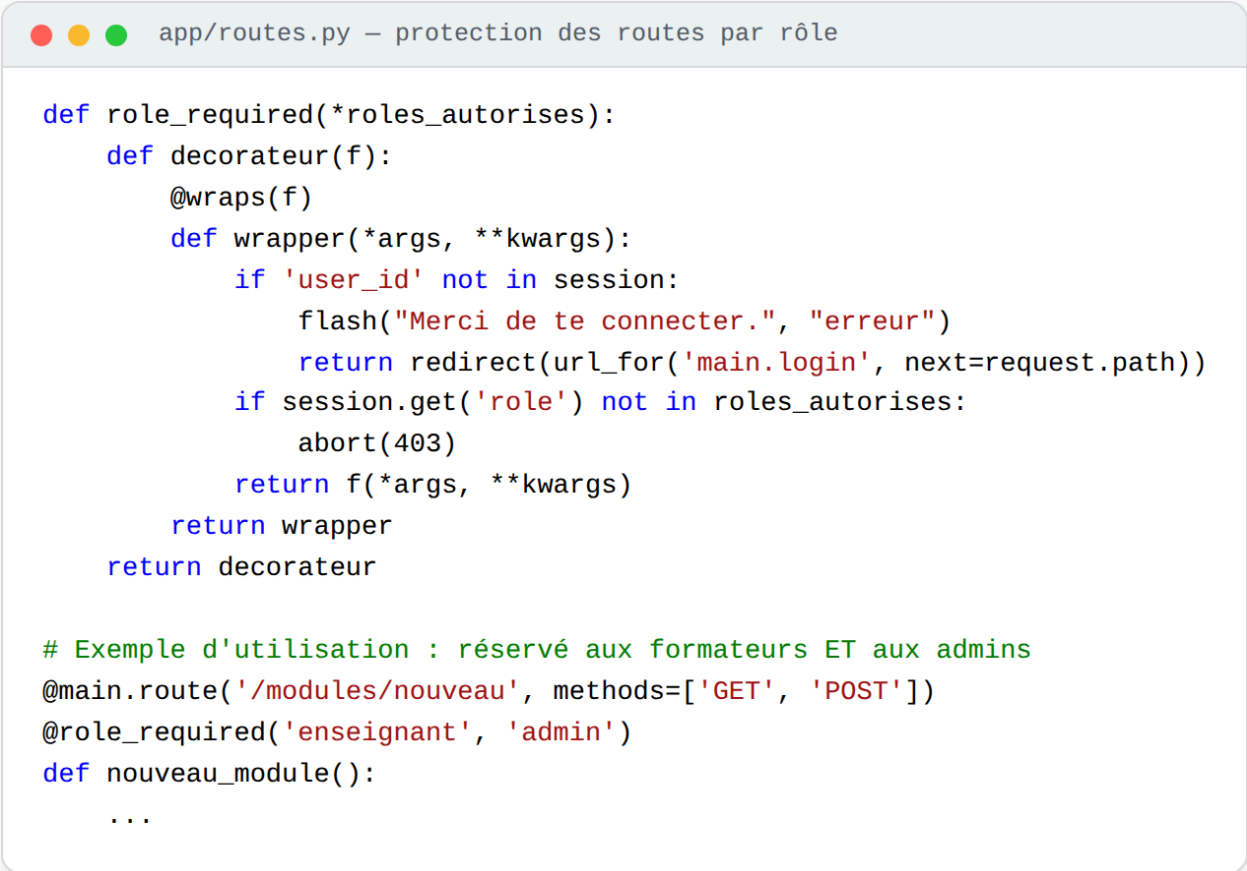
Figure 5.2 — Suppression en deux temps : confirmation puis exécution.

5.2 Authentification et cloisonnement des droits

L'authentification repose sur le mécanisme de session de Flask. À la connexion, l'identifiant et le rôle de l'utilisateur sont placés en session, dans un cookie signé cryptographiquement par la clé secrète de l'application : un utilisateur ne peut donc pas modifier son propre rôle en éditant le cookie côté navigateur.

Les mots de passe ne sont jamais stockés en clair. Le modèle Utilisateur expose deux méthodes, `set_password` et `check_password`, qui s'appuient sur les fonctions de hachage de Werkzeug : seule l'empreinte est enregistrée en base, et la vérification consiste à comparer l'empreinte du mot de passe saisi à celle qui est stockée.

Le contrôle des droits est implémenté par deux décorateurs Python écrits pour le projet. Le premier, `login_required`, redirige vers la page de connexion tout visiteur non authentifié. Le second, `role_required`, prend en paramètre la liste des rôles autorisés et renvoie une erreur 403 si le rôle en session n'y figure pas.



```

app/routes.py — protection des routes par rôle

def role_required(*roles_autorises):
    def decorateur(f):
        @wraps(f)
        def wrapper(*args, **kwargs):
            if 'user_id' not in session:
                flash("Merci de te connecter.", "erreur")
                return redirect(url_for('main.login', next=request.path))
            if session.get('role') not in roles_autorises:
                abort(403)
            return f(*args, **kwargs)
        return wrapper
    return decorateur

# Exemple d'utilisation : réservé aux formateurs ET aux admins
@main.route('/modules/nouveau', methods=['GET', 'POST'])
@role_required('enseignant', 'admin')
def nouveau_module():
    ...

```

Figure 5.3 — Décorateur `role_required` et son application à une route réservée aux enseignants et aux administrateurs.

Ce point mérite d'être souligné, car il constitue la différence entre une interface qui paraît protégée et une application qui l'est réellement. Masquer un bouton dans un gabarit ne protège rien : la route reste accessible à qui en connaît l'adresse. Ici, le contrôle est appliqué à l'entrée de chaque route sensible, côté serveur. Un apprenti qui saisirait directement l'adresse d'une page d'administration reçoit une erreur 403, que le bouton correspondant lui ait été affiché ou non.

5.3 Gestion des comptes utilisateurs

L'application permet de créer, supprimer et réinitialiser les comptes, dans des limites strictement définies par le rôle de celui qui agit. Une fonction dédiée détermine si le compte courant a le droit d'intervenir sur un compte cible : un enseignant ne peut agir que sur des comptes d'apprentis, un administrateur sur les comptes d'apprentis et d'enseignants.



```

app/routes.py – cloisonnement des droits sur les comptes

def _autorise_sur_compte(compte):
    role_courant = session.get('role')
    return (
        (role_courant == 'enseignant' and compte.role == 'apprenti')
        or (role_courant == 'admin' and compte.role in ('apprenti', 'enseignant'))
    )
    # Un formateur ne peut jamais agir sur un compte formateur ou admin.
    # Un compte admin ne se crée que via creer_compte_admin.py, en local
    # : aucune route web ne permet de fabriquer un compte admin.

@main.route('/comptes/<int:compte_id>/supprimer', methods=['POST'])
@role_required('enseignant', 'admin')
def supprimer_compte(compte_id):
    compte = Utilisateur.query.get_or_404(compte_id)
    if not _autorise_sur_compte(compte):
        abort(403)
    ...

```

Figure 5.4 — Cloisonnement des droits sur les comptes : un enseignant ne peut jamais agir sur un compte enseignant ou administrateur.

Un point complète ce dispositif : aucune route de l'application ne permet de créer un compte administrateur. Ce type de compte est créé exclusivement par un script exécuté en local sur le serveur. Le raisonnement est le suivant : le rôle administrateur donne accès à l'ensemble des comptes et au journal d'activité ; si une route web permettait d'en créer un, il suffirait d'une faille dans cette route pour compromettre l'ensemble. En le rendant impossible depuis le web, on supprime la surface d'attaque correspondante.

5.4 Fichiers joints : téléversement, classement, stockage

Chaque fiche module peut recevoir des photos, des vidéos et des documents. Le traitement d'un fichier téléversé suit trois étapes :

- 1. Détermination du type** à partir de l'extension : le fichier est rangé dans l'une des trois catégories et sera affiché dans la galerie correspondante. Une extension non reconnue conduit à ignorer le fichier plutôt qu'à le stocker sans classement.
- 2. Assainissement du nom d'origine** par la fonction `secure_filename`, qui retire les caractères susceptibles de poser problème au système de fichiers, notamment les séquences permettant de remonter dans l'arborescence.
- 3. Attribution d'un nom de stockage unique**, construit à partir de l'identifiant du module, d'une valeur aléatoire et du nom assaini. Deux fichiers portant le même nom d'origine ne peuvent donc pas s'écraser mutuellement.


```

app/routes.py – enregistrement des fichiers joints

def _enregistrer_fichiers(module, fichiers):
    ajoutes = 0
    for fichier in fichiers:
        if not fichier or not fichier.filename:
            continue
        type_f = _type_fichier(fichier.filename) # photo / video / document
        if not type_f:
            continue
        nom_original = secure_filename(fichier.filename)
        nom_stocke = f"m{module.id}_{os.urandom(4).hex()}_{nom_original}"
        chemin = os.path.join(current_app.config['UPLOAD_FOLDER'], nom_stocke)
        fichier.save(chemin)
        taille = os.path.getsize(chemin)
        db.session.add(FichierJoint(
            module_id=module.id, nom_original=nom_original,
            nom_stocke=nom_stocke, type_fichier=type_f, taille_octets=taille,
        ))
        ajoutes += 1
    if ajoutes:
        db.session.commit()
        _journaliser('ajout_fichier', cible=f"{ajoutes} fichier(s)", module_id=module.id)
    return ajoutes

```

Figure 5.5 — Enregistrement des fichiers joints : classement par type, nom de stockage unique et journalisation.

Le nom d'origine est conservé en base afin d'être réaffiché à l'utilisateur : la contrainte technique d'unicité n'est donc pas répercutée sur l'expérience d'utilisation.

5.5 Recherche et espace de consultation

L'espace apprenti propose une recherche par nom de module. Les résultats affichent, pour chaque module correspondant, un aperçu du nombre de photos, de vidéos et de documents disponibles, de manière à ce que l'apprenti sache avant d'ouvrir la fiche s'il y trouvera le schéma qu'il cherche. Lorsque aucun module ne correspond, un message explicite est affiché plutôt qu'une liste vide.

app/routes.py – liste des modules

```
# Affiche la liste de tous les modules enregistrés en base
@main.route('/modules')
def liste_modules():
    modules = Module.query.all() # récupère tous les modules de la base
    return render_template('modules_liste.html', modules=modules)
```

Figure 5.6 — Route de consultation du catalogue des modules.

5.6 Journalisation et tableaux de bord

Toute action significative, connexion, création, modification, suppression, ajout de fichier, opération sur un compte, consultation d'une fiche, donne lieu à l'écriture d'une ligne de journal. La fonction correspondante enregistre l'horodatage, l'identifiant et le rôle de l'auteur, la nature de l'action et l'élément concerné.

app/routes.py – journalisation des actions

```
def _journaliser(action, cible=None, module_id=None):
    """Chaque connexion, création, modification ou suppression est
    enregistrée. Les infos utilisateur sont dupliquées pour que le
    journal reste lisible même si le compte est supprimé plus tard."""
    entree = JournalActivite(
        identifiant_utilisateur=session.get('identifiant', 'inconnu'),
        role_utilisateur=session.get('role', 'inconnu'),
        action=action, cible=cible, module_id=module_id,
    )
    db.session.add(entree)
    db.session.commit()

# Appelé notamment à chaque connexion et à chaque consultation de fiche,
# ce qui permet ensuite un classement réel des modules les plus vus.
```

Figure 5.7 — Fonction de journalisation : les informations de l'auteur sont dupliquées pour que le journal reste lisible dans le temps.

Ce journal alimente deux tableaux de bord. Celui de l'enseignant présente le volume de la base documentaire et signale les modules incomplets, c'est-à-dire dépourvus de description ou de fichier joint : c'est un outil de travail, qui indique où l'effort de documentation doit porter. Celui de l'administrateur ajoute des indicateurs d'usage : répartition des modules par catégorie, activité sur les derniers jours, classement des modules les plus consultés et des comptes les plus actifs.

La journalisation des consultations donne à ce dernier point une valeur particulière : le classement des modules les plus consultés ne repose pas sur une estimation mais sur des consultations réellement enregistrées. À l'échelle d'une année, cet indicateur permettrait d'identifier les modules sur lesquels les apprentis reviennent le plus, et donc ceux dont la documentation mérite d'être approfondie en priorité.

Infrastructure serveur et réseau

L'application est hébergée sur un serveur physique dédié de l'établissement, un Lenovo ThinkSystem ST50 portant le nom de machine bts-blanc. Ce chapitre décrit son installation, son durcissement et l'architecture réseau qui le dessert.

Il rend compte également de la reconstruction du réseau autour d'OPNsense, consécutive à la

panne du routeur initialement prévu.

L'infrastructure a été montée en deux temps : d'abord le serveur applicatif, préparé pendant l'immobilisation du matériel réseau, puis le réseau lui-même, reconstruit après la panne du routeur initialement prévu.

6.1 Matériel et découpage du stockage

Le serveur est équipé d'un SSD Kingston d'environ 223 Go. Le stockage a été configuré en LVM lors de l'installation. Ce gestionnaire de volumes logiques introduit une couche d'abstraction entre les partitions physiques et les systèmes de fichiers : il devient possible de redimensionner un volume, voire d'y agréger un disque supplémentaire, sans réinstaller le système. Dans le contexte de ce projet, où les fichiers joints, en particulier les vidéos, peuvent occuper un volume croissant et difficile à anticiper, cette souplesse constitue une précaution utile.

Le système a été installé en édition standard plutôt qu'en version minimale, afin de disposer des outils d'administration courants sur la machine sans avoir à les ajouter un par un, un choix qui privilégie le confort de diagnostic sur un serveur destiné à être exploité par plusieurs personnes.

6.2 Administration et sécurisation système

Le serveur est administré exclusivement en ligne de commande, à distance, par SSH. L'option correspondante a été activée dès l'installation du système, ce qui a permis de piloter la machine sans y raccorder d'écran ni de clavier après sa mise en service initiale.

L'administration s'effectue au moyen d'un compte utilisateur standard créé à l'installation, l'élévation de privilèges passant par `sudo`. Travailler ainsi plutôt que directement sous le compte `root` présente deux bénéfices : les actions d'administration restent attribuables à un compte identifié dans les journaux du système, et une commande destructrice lancée par erreur ne s'exécute pas avec les pleins pouvoirs par défaut.

Le renforcement de cet accès constitue la première évolution à apporter au serveur : passage à une authentification par clé publique et interdiction explicite de la connexion du compte `root` dans la configuration du service SSH. Ces deux mesures sont documentées et comprises, mais n'ont pas été appliquées dans le temps du projet.

6.3 Déploiement et durcissement de MariaDB

Le moteur MariaDB a été installé depuis les dépôts de la distribution, puis sécurisé au moyen de l'assistant `mariadb-secure-installation`. Cet assistant applique quatre mesures qui constituent le minimum attendu avant toute mise en service :

- Définition d'un mot de passe pour le compte `root` du moteur, qui n'en possède pas par défaut.
- Suppression des comptes anonymes, créés à l'installation, qui permettent une connexion sans identifiant.
- Interdiction de la connexion du compte `root` depuis une autre machine que le serveur lui-même.
- Suppression de la base de données de test, accessible en écriture à tout utilisateur.

L'application ne se connecte jamais avec le compte root. Un utilisateur applicatif dédié a été créé, dont les droits sont limités à la seule base knx_doc. Cette séparation applique le principe du moindre privilège : si les identifiants de connexion de l'application venaient à être compromis, la portée de la compromission resterait circonscrite à cette base, sans possibilité d'agir sur le moteur ni sur d'autres bases.

6.4 Architecture réseau : OPNsense et segmentation

Le réseau du projet a été reconstruit autour d'un pare-feu et routeur OPNsense installé sur une machine physique dédiée, en remplacement du routeur tombé en panne. Une segmentation en VLAN selon la norme 802.1Q a été mise en place à cette occasion.

Le principe d'une segmentation est d'isoler logiquement plusieurs réseaux sur une même infrastructure physique. Appliqué à ce projet, il permet de séparer le serveur applicatif, les postes enseignants et les postes apprentis : chacun dispose de son propre plan d'adressage, et le pare-feu décide explicitement de ce qui peut circuler entre ces segments.

Les deux équipements et leur rôle

OPNsense, installé sur une machine dédiée à l'adresse 192.168.73.1, assure le routage entre les segments, le filtrage et la distribution automatique des adresses par DHCP au moyen de Dnsmasq. Il porte une interface virtuelle par VLAN.

Le switch TL-SG108PE assure la commutation. Il est relié au pare-feu par un lien unique, le port 1, configuré en trunk 802.1Q, qui transporte les VLAN 10, 20 et 30 étiquetés. Les autres ports sont des ports d'accès : chacun se voit attribuer un PVID, c'est-à-dire le VLAN auquel appartient par défaut le trafic non étiqueté qui y entre. C'est ce mécanisme qui fait qu'un poste ordinaire, qui ne sait rien des VLAN, se retrouve automatiquement dans le bon segment selon la prise sur laquelle il est branché.

La difficulté de cette configuration tient à la cohérence exigée entre les deux équipements : un VLAN déclaré côté pare-feu mais absent du switch, ou un port d'accès dont le PVID ne correspond pas, se traduit par un segment silencieux, sans message d'erreur d'aucun côté.

Plan de segmentation

VLAN	Usage	Adressage	État
10	Serveur Ubuntu, application Flask et base MariaDB	Segment dédié au serveur	Opérationnel
20	Postes enseignants	192.168.20.0/24	Opérationnel
30	Postes apprentis	192.168.30.0/24	Opérationnel
40	Administration réseau (ajout hors périmètre imposé)	Adressage à définir	En cours

Le switch est lui-même administrable et joignable sur 192.168.0.1. Un point d'accès sans fil TL-WA801N complète l'installation à l'adresse 192.168.73.254 ; le DHCP est distribué automatiquement sur les segments serveur, enseignants et apprentis.

Le VLAN 40, destiné à isoler l'administration des équipements réseau du reste du trafic, est un ajout que nous avons engagé au-delà du périmètre demandé. Son adressage reste à définir : il est présenté ici comme en cours, et non comme opérationnel.

Tests et recette

La qualification du système s'appuie sur un cahier de recette de 30 points de contrôle, couvrant les fonctionnalités applicatives, l'interface, la gestion des comptes et des fichiers, la sécurité, l'infrastructure réseau et la gestion de version.

Ce chapitre expose la démarche suivie et, surtout, l'état exact du système au moment du

rendu, sans arrondir dans un sens ni dans l'autre.

La distinction entre vérification en environnement isolé et validation en conditions réelles structure l'ensemble de ce chapitre. Elle explique pourquoi certaines fonctionnalités développées ne sont pas présentées comme validées.

7.1 Démarche de validation

Nous distinguons rigoureusement deux niveaux de validation, et cette distinction structure l'ensemble du présent chapitre.

- La **vérification en environnement isolé** consiste à rejouer un scénario dans un environnement de test cloisonné, avec une base jetable, indépendamment du serveur réel. Elle apporte une garantie sur la logique du code.
- La **validation en conditions réelles** consiste à exécuter le même scénario dans un navigateur, sur le serveur déployé, avec la base de données réelle. Elle seule permet de conclure que la fonctionnalité fonctionne pour l'utilisateur final.

Nous avons tenu à ne jamais présenter le premier niveau comme équivalent au second. Cette prudence s'est révélée fondée : le premier passage en conditions réelles a immédiatement fait apparaître un défaut que les tests isolés n'avaient pas révélé, décrit au chapitre suivant.

7.2 Synthèse du cahier de recette

État	Nombre	Signification
Non implémenté	2	Fonctionnalité non développée : il n'y a rien à tester (sauvegardes automatiques, comportement en cas d'indisponibilité de la base)
Vérifié en environnement isolé	à confirmer	Logique validée par un test automatisé, validation en conditions réelles restant à effectuer
À tester en conditions réelles	à confirmer	Points restant à exécuter et à cocher dans le cahier de recette

Deux points sont d'emblée identifiés comme non implémentés, puisqu'il n'y a rien à y tester : les sauvegardes automatiques de la base et le comportement de l'application en cas d'indisponibilité de celle-ci. Les vingt-huit autres sont à exécuter et à cocher au fur et à mesure dans le cahier de recette joint en annexe, qui fait foi. Nous n'affichons volontairement aucun taux de conformité global : un pourcentage calculé sur des points non encore exécutés ne signifierait rien.

7.3 État factuel du système au 8 septembre 2026

Fonctionnel et conforme au cahier des charges

- Gestion complète des fiches modules : création, consultation, modification, suppression avec confirmation.
- Interface différenciée par rôle et charte graphique cohérente sur l'ensemble des pages.
- Connexion sécurisée à la base de données MariaDB par l'intermédiaire de l'ORM.

Développé et vérifié en environnement isolé, à valider en conditions réelles

- Authentification et gestion des sessions pour les trois rôles.
- Cloisonnement des droits, y compris le refus d'une tentative d'élévation de privilège.
- Gestion des comptes utilisateurs et restrictions associées au rôle courant.
- Téléversement, classement et affichage des fichiers joints.
- Journalisation des actions et tableaux de bord.

En cours au moment du rendu

- Le déploiement de l'application sur le serveur : l'infrastructure système est prête et sécurisée, l'installation de l'application elle-même est en cours de finalisation.
- Les tests d'infrastructure réseau dans la configuration OPNsense, désormais possibles depuis le remplacement du routeur.

Non réalisé

- Les sauvegardes automatiques de la base de données : seule fonctionnalité du cahier des charges entièrement non développée.
- Le chiffrement des échanges en HTTPS.

Le détail des 30 points de contrôle, avec leur mode opératoire et l'emplacement destiné à consigner le résultat obtenu, figure dans le cahier de recette joint en annexe.

Difficultés et solutions

Cinq difficultés significatives ont jalonné le projet. Elles sont exposées ici avec la démarche de diagnostic suivie, car cette démarche constitue elle-même une part du travail réalisé.

Trois d'entre elles relèvent d'un même écart : celui qui sépare un environnement de

développement d'un environnement de production réel.

Chaque difficulté est présentée selon la même trame : le symptôme observé, l'analyse de la cause, la résolution appliquée et l'enseignement qui en a été tiré pour la suite du projet.

8.1 Évolution du schéma de base sans outil de migration

Symptôme

Lors du premier essai en conditions réelles, après l'ajout des fonctionnalités de fichiers joints et de journal d'activité, l'application s'est interrompue sur l'erreur suivante : `sqlalchemy.exc.OperationalError : Unknown column 'fichiers_joints.taille_octets' in 'field list'`.

Analyse

Les tables avaient été créées lors d'une exécution antérieure du script d'initialisation, avant l'ajout de la colonne `taille_octets` au modèle. Or la méthode utilisée par ce script, `db.create_all()`, ne crée que les tables absentes : elle ne compare pas la structure des tables existantes avec celle des modèles Python et ne modifie jamais une table déjà créée. Le code et la base réelle s'étaient donc désynchronisés silencieusement, sans qu'aucune erreur n'apparaisse au démarrage.

Résolution

Deux instructions `ALTER TABLE` ont été exécutées manuellement pour ajouter les colonnes manquantes, sans perte des données déjà saisies. L'incident a été corrigé en quelques minutes, mais il révèle une limite réelle de l'approche : à chaque évolution du modèle, la base doit être mise à jour manuellement, opération non tracée et sujette à l'oubli. La solution durable consiste à introduire un outil de migration tel qu'Alembic, qui génère et applique les scripts de modification de schéma en les versionnant au même titre que le code.

Enseignement

Cet incident illustre surtout la valeur du test en conditions réelles : il n'aurait pas pu être détecté par un test isolé, puisque celui-ci recrée systématiquement une base vierge à partir des modèles, donc toujours parfaitement synchronisée. C'est précisément la raison pour laquelle nous refusons de considérer une fonctionnalité comme validée sur la seule foi d'un test isolé.

8.2 Incompatibilité de l'environnement Python entre macOS et Ubuntu

Symptôme

Lors des premiers essais de transfert du projet vers le serveur, l'environnement virtuel Python copié depuis le poste de développement refusait de fonctionner.

Analyse

Un environnement virtuel ne contient pas uniquement du code Python : il embarque des binaires et des bibliothèques compilés pour le système et l'architecture de la machine sur laquelle il a été créé. Un environnement construit sous macOS sur processeur ARM contient donc des fichiers inutilisables sur un serveur Ubuntu en architecture x86-64.

Résolution

Le dossier de l'environnement virtuel a été exclu du dépôt Git et n'est jamais transféré. Sur chaque machine, il est recréé localement, puis les dépendances sont installées à partir du fichier `requirements.txt`, qui liste les paquets et leurs versions de manière portable. Cette pratique garantit que chaque environnement est construit pour la machine qui l'exécute, tout en assurant que les versions installées sont identiques de part et d'autre.

8.3 Accessibilité réseau du serveur applicatif

Symptôme

L'application, démarrée sur le serveur, répondait correctement depuis le serveur lui-même mais restait inaccessible depuis les autres postes du réseau.

Analyse

Par défaut, le serveur de développement de Flask n'écoute que sur l'adresse de bouclage `127.0.0.1`, qui ne désigne que la machine locale. Les requêtes provenant d'autres adresses ne parviennent donc jamais à l'application. Il ne s'agit pas d'un défaut mais d'une mesure de prudence : cette configuration évite d'exposer involontairement sur un réseau une application en cours de développement.

Résolution

L'écoute a été portée sur `0.0.0.0`, ce qui signifie « toutes les interfaces réseau de la machine », et le port correspondant a été autorisé dans le pare-feu du serveur. Le mode de débogage a par ailleurs été désactivé pour cette exposition : laissé actif, il affiche en cas d'erreur une trace détaillée du code source, et permet même l'exécution de code depuis le navigateur.

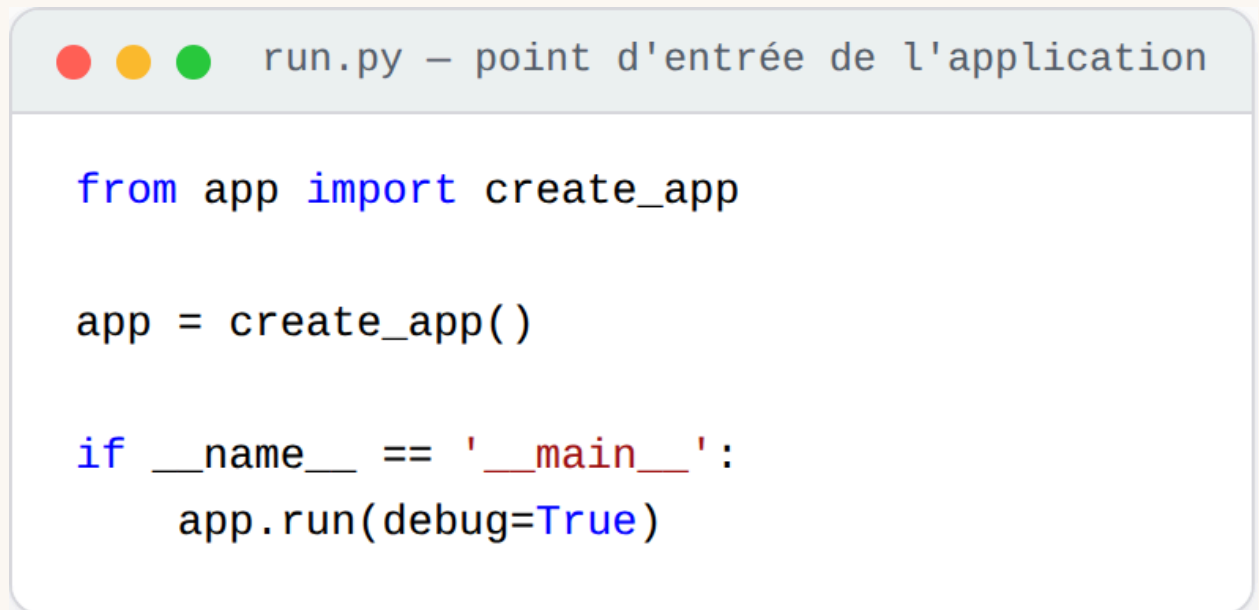


Figure 8.1 — Point d'entrée de l'application.

8.4 Panne matérielle du routeur et diagnostic par élimination

Symptôme

Lors de la configuration du routeur TP-Link ER605 destiné au projet, plus aucun port LAN ne délivrait de lien réseau : aucune diode d'activité côté adaptateur, aucune adresse obtenue par un poste raccordé.

Démarche de diagnostic

Le diagnostic a procédé par élimination successive des causes possibles, de la plus probable à la moins probable :

1. Remplacement des câbles par des cordons vérifiés, sans effet.
2. Changement de port sur le boîtier, sans effet, aucun voyant de lien sur aucun port.
3. Réinitialisation aux paramètres d'usine, à plusieurs reprises, le voyant système, d'abord éteint, s'est remis à clignoter normalement après un dernier essai, sans que les ports LAN redeviennent fonctionnels.
4. Contrôle et correction de l'alimentation, initialement non conforme aux caractéristiques du routeur, sans rétablissement des ports.

Conclusion et résolution

Le retour à la normale du voyant système, alors que les ports restaient inertes, oriente vers une défaillance du circuit de commutation interne, vraisemblablement consécutive à l'alimentation incorrecte utilisée avant sa correction. Cette panne dépassant le cadre d'une erreur de configuration réparable, elle a été remontée à l'enseignant pour remplacement du matériel.

Le remplacement a été effectué avec une solution différente du matériel d'origine : un pare-feu et routeur OPNsense installé sur une machine physique dédiée, avec segmentation par VLAN. En attendant ce remplacement, le travail s'est poursuivi sur deux fronts indépendants du routeur en panne : le développement applicatif en local et la préparation du serveur, raccordé directement au réseau filaire déjà fonctionnel de la salle. Cette organisation a permis de ne pas immobiliser le projet pendant l'indisponibilité du matériel.

8.5 Mise en place tardive de la gestion de version

Symptôme

Alors que le cahier des charges impose un historique Git complet, le dossier de travail n'était relié à aucun dépôt local à un stade avancé du projet.

Analyse

Le développement avait démarré directement dans un dossier de travail, la mise en place du versionnage étant repoussée au profit de l'avancement fonctionnel. Cette dette s'est traduite par une absence de points de restauration : aucune possibilité de revenir à un état antérieur en cas de régression, et aucune trace de l'évolution du projet.

Résolution

Le dépôt a été initialisé, accompagné d'un fichier `.gitignore` excluant les éléments qui n'ont pas à être versionnés : l'environnement virtuel, les fichiers compilés, les fichiers téléversés par l'application et les fichiers propres au système d'exploitation. L'historique a été reconstitué en commits thématiques suivant la convention `feat: / fix: / docs:`, et le dépôt relié au dépôt distant.

Enseignement

C'est la difficulté dont l'enseignement est le plus net : l'initialisation du dépôt aurait dû constituer la toute première action du projet, avant même la première ligne de code. Le coût en aurait été de quelques minutes, contre plusieurs heures de reconstitution a posteriori et une période où le travail n'était protégé par aucune sauvegarde.

Gestion de projet

Le projet a été rythmé par les revues du calendrier pédagogique, de l'analyse du besoin jusqu'à la soutenance finale. Ce chapitre rend compte du suivi, des conventions de développement adoptées et, surtout, des écarts entre ce qui était prévu et ce qui a été réalisé.

L'analyse de ces écarts constitue la partie la plus instructive du chapitre : elle explique où le temps

est réellement passé.

Le suivi s'est appuyé sur les quatre revues du calendrier pédagogique. L'écart le plus important tient à un événement non planifiable : la panne du matériel réseau, qui a redistribué une partie de la charge du projet.

9.1 Jalons et suivi

Le projet s'est déroulé sur huit jours, du 31 août au 8 septembre 2026, rythmé par quatre jalons imposés. Les cinq échéances ont toutes été tenues.

Jalon	Date	Contenu	État
R1	31/08/2026	Analyse du besoin, identification des deux publics, orientations techniques	Tenue
R2	01/09/2026	Cahier des charges, choix Flask et MariaDB, contraintes, planning	Tenue
R3	02/09/2026	Démonstration et tests : socle applicatif, fonctionnalités transverses, constat de la panne du routeur	Tenue
—	avant le 08/09	Finalisation du développement et reconstruction de l'infrastructure réseau	Tenue
RF	08/09/2026	Recette, déploiement, remise du dossier et soutenance	Tenue

Cette durée très courte est la donnée qui explique le mieux la forme de ce rapport. Sur huit jours, chaque demi-journée consacrée à un imprévu est une demi-journée retirée à autre chose : c'est la raison pour laquelle certaines fonctionnalités sont assumées comme non réalisées plutôt que livrées à moitié.

9.2 Gestion de version et conventions

Le code est versionné sous Git et hébergé sur un dépôt distant. Les messages de commit suivent la convention des commits conventionnels, qui préfixe chaque message par la nature de la modification apportée :

```
feat: authentication et gestion des rôles (admin/formateur/apprenti)
feat: gestion des comptes, upload de fichiers et recherche apprenti
fix: clé secrète de session définitive et écoute réseau 0.0.0.0
docs: cahier de recette et bilan de projet
```

L'intérêt de cette convention dépasse la lisibilité : elle oblige à découper le travail en modifications cohérentes. Un commit qui ne peut pas être décrit par un préfixe unique est généralement un commit qui mélange plusieurs sujets et qu'il faudrait scinder.

9.3 Analyse des écarts entre le prévu et le réalisé

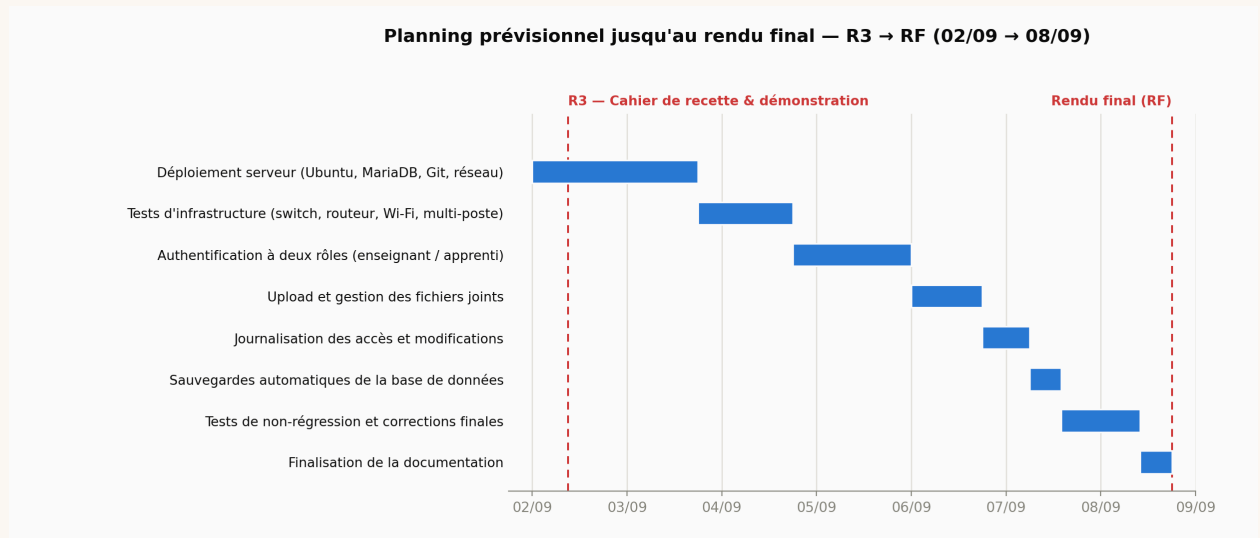


Figure 9.1 — Planning des tâches restantes jusqu'au rendu final.

Trois écarts principaux séparent le prévisionnel du réalisé.

- La **panne du routeur** a immobilisé la partie réseau du projet, puis conduit à reconstruire l'infrastructure sur une base différente. Cet imprévu a consommé du temps qui était prévu pour les fonctionnalités secondaires, mais a produit une infrastructure plus riche que celle envisagée initialement.
- À l'inverse, l'**authentification, la gestion des comptes, la journalisation et les tableaux de bord** ont été développés en avance sur le planning, et le tableau de bord d'administration dépasse le périmètre initial du cahier des charges.
- Les **sauvegardes automatiques**, planifiées en fin de parcours, n'ont pas pu être traitées. C'est la conséquence directe des deux points précédents : le temps gagné sur les fonctionnalités a été absorbé par l'infrastructure.

Avec le recul, la principale leçon de gestion porte sur l'ordre des travaux. Le versionnage et le déploiement, traités tardivement, auraient dû être mis en place tôt, sur une application encore minimale. Déployer une application de dix routes est une opération simple ; déployer une application terminée, sous contrainte de délai, concentre en une seule étape tous les problèmes d'environnement, de configuration et de réseau.

Compétences BTS CIEL mobilisées

Le tableau qui suit met en regard les compétences du référentiel du BTS CIEL et les éléments du projet qui en apportent la preuve, avec le renvoi au chapitre correspondant.

Il est conçu pour que chaque compétence puisse être vérifiée dans le rapport, et non simplement

affirmée.

Chaque ligne du tableau renvoie au chapitre où la compétence est démontrée, de manière à ce que la preuve puisse être vérifiée directement dans le corps du rapport.

Domaine de compétence	Preuve apportée par le projet	Voir
Analyser un besoin et formaliser des exigences	Analyse du besoin documentaire du banc KNX, identification des deux publics, rédaction du cahier des charges et du cahier de recette en 30 points	Ch. 1, 2
Concevoir une architecture logicielle et un modèle de données	Application factory, séparation modèles / contrôleurs / vues, modélisation des quatre tables et arbitrages de normalisation du journal	Ch. 3
Développer une application web	Développement en Python et Flask : routes, formulaires, gabarits Jinja2, gestion de sessions, traitement des fichiers téléversés	Ch. 5
Concevoir et administrer une base de données	Modélisation relationnelle, accès par ORM, requêtes paramétrées, compte applicatif dédié, correction manuelle d'un schéma désynchronisé	Ch. 3, 6, 8
Sécuriser une application	Hachage des mots de passe, sessions signées, contrôle d'accès par rôle appliqué côté serveur, cloisonnement des opérations sur les comptes, journalisation, protection contre l'injection SQL par conception	Ch. 5
Installer et sécuriser un système serveur	Installation d'Ubuntu Server, partitionnement LVM, administration à distance par SSH sans connexion root, durcissement de MariaDB	Ch. 6
Configurer et diagnostiquer une infrastructure réseau	Diagnostic d'une panne matérielle par élimination méthodique, déploiement d'un pare-feu OPNsense, segmentation en VLAN 802.1Q, configuration d'un switch administrable	Ch. 6, 8
Qualifier un système par une démarche de recette	Cahier de recette de 30 points, distinction explicite entre vérification isolée et validation en conditions réelles, traçabilité des anomalies	Ch. 7
Conduire un projet et communiquer techniquement	Respect des jalons R1 à RF, versionnage Git avec conventions de commit, production du cahier des charges, du cahier de recette et du présent rapport	Ch. 9

10.1 Ce que le projet m'a concrètement appris

Au-delà du référentiel, six apprentissages précis ressortent de ces huit jours.

Apprentissage		Contenu
1	Équipements réseau	Distinguer en pratique ce que fait un point d'accès, un switch et un routeur ou pare-feu, une distinction qui reste théorique tant qu'on n'a pas eu à câbler et configurer les trois.
2	VLAN 802.1Q	Différence entre port trunk et port d'accès, rôle du PVID, et surtout l'exigence de cohérence entre la configuration du switch et celle du pare-feu.
3	Pare-feu open source	Installation d'OPNsense, distinction des interfaces WAN et LAN, création d'interfaces virtuelles par VLAN, mise en service du DHCP.
4	Administration Linux	Ubuntu Server, gestion des paquets, configuration réseau, administration à distance par SSH.
5	Sécurité des bases de données	Durcissement de MariaDB et application concrète du principe de moindre privilège à un compte applicatif.
6	Gestion de projet	Prioriser sous contrainte de temps et arbitrer entre les livrables imposés et les ajouts que l'on aurait voulu traiter.

Conclusion

Le socle applicatif est fonctionnel et conforme au cahier des charges : gestion complète des fiches modules, interface différenciée selon le rôle, connexion sécurisée à la base de données.

Ce chapitre dresse le bilan de ce qui a été livré, de ce qui reste à confirmer et de ce qui manque,

puis le bilan personnel que je tire de ce projet.

La conclusion se veut factuelle : elle rend compte de ce qui fonctionne, de ce qui reste à confirmer et de ce qui n'a pas été fait, sans arrondir dans un sens ni dans l'autre.

11.1 Bilan du projet

Les fonctionnalités transverses identifiées comme manquantes lors de la revue précédente, authentification, gestion des comptes, fichiers joints, journalisation, tableaux de bord, ont été développées, et leur logique de permissions a été vérifiée par des tests automatisés en environnement isolé. Leur validation définitive en conditions réelles reste à mener à son terme.

L'infrastructure du serveur est installée et sécurisée. Le réseau du projet, interrompu par la panne du routeur, a été reconstruit autour d'un pare-feu OPNsense avec segmentation par VLAN. Le déploiement de l'application sur ce serveur est en cours de finalisation.

Deux points du cahier des charges n'ont pas été traités : les sauvegardes automatiques de la base de données et le chiffrement des échanges. Ce sont des manques réels, dont nous rendons compte comme tels.

Ce projet a produit un système utilisable et une infrastructure saine, dans un contexte marqué par un délai court et une panne matérielle. Nous ne le présentons pas comme achevé : nous le présentons pour ce qu'il est, avec ce qui fonctionne, ce qui reste à confirmer et ce qui manque.

11.2 Bilan personnel

Ce projet m'a d'abord fait mesurer concrètement l'écart entre un environnement de développement et un environnement de production. Des éléments qui ne posent aucune difficulté sur un poste de travail, un environnement virtuel, une configuration réseau par défaut, un schéma de base de données créé une fois pour toutes, demandent une attention entièrement différente dès lors qu'ils sont transposés sur un serveur partagé, accessible à d'autres utilisateurs. Les trois incidents les plus instructifs du projet relèvent tous de cet écart.

Mon alternance à la Banque de France a nourri ma façon d'aborder la partie sécurité. Dans un environnement où la traçabilité et le cloisonnement des accès sont des exigences quotidiennes, on prend l'habitude de se demander non pas si une protection est visible, mais si elle est effective. C'est ce qui m'a conduit à vérifier que le contrôle des droits était bien appliqué côté serveur, et à concevoir le journal d'activité pour qu'il reste exploitable même après la suppression d'un compte.

J'en retire enfin une leçon de méthode. Les deux difficultés qui nous ont coûté le plus de temps, le versionnage mis en place tardivement et le déploiement traité en fin de parcours, n'étaient pas des problèmes techniques difficiles, mais des tâches structurantes repoussées au profit de l'avancement fonctionnel. Sur un prochain projet, l'ordre serait différent : versionner et déployer une application minimale dès les premiers jours, puis la faire grandir, plutôt que de reporter ces étapes à la fin.

Cette expérience conforte mon orientation vers l'administration et la sécurisation des infrastructures, domaine dans lequel je souhaite poursuivre.

Glossaire technique

Les termes techniques employés dans ce rapport sont définis ci-après, afin que le document reste lisible par un lecteur qui ne serait pas spécialiste de tous les domaines qu'il couvre.

Les définitions sont volontairement courtes et rédigées en langage courant, de manière à rester utiles à un lecteur non spécialiste du domaine concerné.

CRUD

Create, Read, Update, Delete. Les quatre opérations élémentaires appliquées à une donnée persistante : créer, consulter, modifier, supprimer.

DHCP

Dynamic Host Configuration Protocol. Protocole attribuant automatiquement une configuration réseau à une machine qui se connecte.

Flask

Micro-framework web écrit en Python. Il fournit le minimum nécessaire pour traiter des requêtes HTTP et produire des pages, les fonctions supplémentaires étant ajoutées par extensions.

Injection SQL

Attaque consistant à insérer des instructions SQL dans une saisie utilisateur afin de détourner une requête. L'usage de requêtes paramétrées, comme le fait un ORM, la rend inopérante.

Jinja2

Moteur de gabarits utilisé par Flask. Il permet d'insérer des données dynamiques dans des pages HTML et de faire hériter plusieurs pages d'une structure commune.

KNX

Standard international ouvert (ISO/IEC 14543) de contrôle et d'automatisation des bâtiments, reposant sur un bus de communication entre modules.

LVM

Logical Volume Manager. Couche de gestion du stockage sous Linux permettant de redimensionner des volumes ou d'y agréger des disques sans réinstaller le système.

OPNsense

Distribution open source de pare-feu et de routage, installable sur une machine banalisée.

ORM

Object-Relational Mapping. Technique consistant à manipuler les données d'une base relationnelle sous forme d'objets du langage de programmation, la traduction en SQL étant assurée par la bibliothèque.

secure_filename

Fonction de la bibliothèque Werkzeug qui assainit un nom de fichier téléversé en retirant les caractères susceptibles de poser problème au système de fichiers.

SSH

Secure Shell. Protocole chiffré permettant d'administrer une machine à distance en ligne de commande.

VLAN

Virtual Local Area Network. Réseau local virtuel permettant de segmenter logiquement une infrastructure physique unique en plusieurs réseaux isolés (norme IEEE 802.1Q).

Werkzeug

Bibliothèque sur laquelle repose Flask. Elle fournit notamment les fonctions de hachage et de vérification des mots de passe utilisées dans ce projet.

Annexes

Deux documents complètent ce rapport et sont joints au dossier : le cahier des charges validé lors des premières revues, et le cahier de recette détaillant les 30 points de contrôle.

Les deux annexes constituent les documents de référence du projet : ce qui était demandé, et la grille qui permet d'en vérifier la réalisation point par point.

Annexe 1 : Cahier des charges

Le cahier des charges validé lors des premières revues du projet est joint au présent rapport. Il détaille les besoins fonctionnels, les contraintes techniques, les contraintes de sécurité et le planning initial.

Annexe 2 : Cahier de recette

Le cahier de recette est joint sous la forme d'un document distinct. Il présente, pour chacun des 30 points de contrôle, le mode opératoire à suivre, le résultat attendu et, lorsqu'elle existe, la preuve sous forme d'extrait de code. Les emplacements destinés au résultat obtenu, au statut et à la date sont à renseigner au fur et à mesure de l'exécution des tests.